

```

1 import torch
2
3 print("CUDA Available:", torch.cuda.is_available())
4 if torch.cuda.is_available():
5     print("Device Name:", torch.cuda.get_device_name(0))

```

CUDA Available: True
Device Name: Tesla T4

1: Environment Setup & Dataset Download

```

1 import os
2 import tarfile
3 import urllib.request
4 import numpy as np
5 import matplotlib.pyplot as plt
6 from PIL import Image
7
8 import torch
9 import torch.nn as nn
10 import torch.nn.functional as F
11 from torch.utils.data import Dataset, DataLoader
12 import torchvision.transforms as T
13
14 # Device Setup
15 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
16 print(f"Using device: {device}")
17
18 # Download Oxford-IIIT Pet Dataset
19 dataset_url_images = "https://www.robots.ox.ac.uk/~vgg/data/pets/data/images.tar.gz"
20 dataset_url_annotations = "https://www.robots.ox.ac.uk/~vgg/data/pets/data/annotations.tar.gz"
21
22 if not os.path.exists("images"):
23     print("Downloading images...")
24     urllib.request.urlretrieve(dataset_url_images, "images.tar.gz")
25     with tarfile.open("images.tar.gz", "r:gz") as tar:
26         tar.extractall(filter='data')
27
28 if not os.path.exists("annotations"):
29     print("Downloading annotations...")
30     urllib.request.urlretrieve(dataset_url_annotations, "annotations.tar.gz")
31     with tarfile.open("annotations.tar.gz", "r:gz") as tar:
32         tar.extractall(filter='data')
33
34 print("Dataset download and extraction complete.")

```

Using device: cuda
Dataset download and extraction complete.

2: Dataset Exploration & 3x3 Visualization

```

1 # Extract all pet class names from image files
2 image_dir = "images"
3 annotation_dir = "annotations/trimaps"
4
5 all_image_files = [f for f in os.listdir(image_dir) if f.endswith(".jpg") and not f.startswith(".")]
6
7 def get_class_name(filename):
8     # Formats 'yorkshire_terrier_12.jpg' -> 'Yorkshire Terrier'
9     parts = filename.rsplit("_", 1)[0].split("_")
10    return " ".join([p.capitalize() for p in parts])
11
12 # Filter valid files that have corresponding trimap annotations
13 valid_files = [f for f in all_image_files if os.path.exists(os.path.join(annotation_dir, f.replace(".jpg", ".png")))]
14
15 # Plot 3x3 Grid
16 np.random.seed(42)
17 sample_indices = np.random.choice(len(valid_files), 9, replace=False)
18
19 fig, axes = plt.subplots(3, 3, figsize=(10, 10))
20 for idx, ax in zip(sample_indices, axes.ravel()):
21     img_name = valid_files[idx]
22     img_path = os.path.join(image_dir, img_name)
23     mask_path = os.path.join(annotation_dir, img_name.replace(".jpg", ".png"))
24
25     img = Image.open(img_path).convert("RGB")
26     mask = Image.open(mask_path)
27     mask_arr = np.array(mask)
28
29     # Process boundary (3) and foreground (1) -> 1, background (2) -> 0
30     binary_mask = np.where((mask_arr == 1) | (mask_arr == 3), 1, 0)

```

```

31
32 ax.imshow(img)
33 ax.imshow(binary_mask, alpha=0.4, cmap="jet")
34 ax.set_title(get_class_name(img_name))
35 ax.axis("off")
36
37 plt.tight_layout()
38 plt.show()

```

Wheaten Terrier



Abyssinian



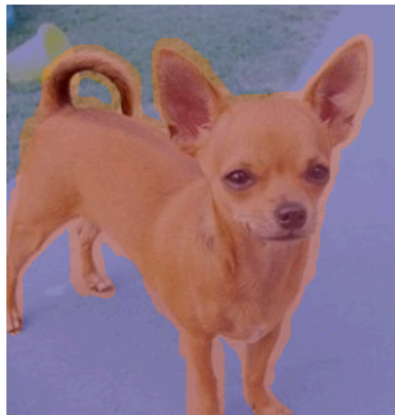
Chihuahua



Miniature Pinscher



Chihuahua



Havanese



Abyssinian



German Shorthaired



Miniature Pinscher



✓ # 3: Base U-Net & Attention U-Net Architectures

```

1 # Step 3: PyTorch Dataset Definition & Train/Val/Test Splits
2 from PIL import Image
3 import torchvision.transforms as T
4
5 # Extract unique breed names and build label mappings
6 classes = sorted(list(set([get_class_name(f) for f in valid_files])))
7 class_to_idx = {cls_name: i for i, cls_name in enumerate(classes)}
8 num_classes = len(classes)
9
10 # 1. Define Dataset Class
11 class PetSegmentationDataset(Dataset):
12     def __init__(self, filenames, img_dir, mask_dir, img_size=(128, 128)):
13         self.filenames = filenames
14         self.img_dir = img_dir
15         self.mask_dir = mask_dir
16         self.img_size = img_size
17
18     def __len__(self):
19         return len(self.filenames)
20
21     def __getitem__(self, idx):
22         fname = self.filenames[idx]
23         img_path = os.path.join(self.img_dir, fname)
24         mask_path = os.path.join(self.mask_dir, fname.replace(".img", ".png"))

```

```

25
26     # Load Image and Resize
27     img = Image.open(img_path).convert("RGB").resize(self.img_size)
28     img_tensor = T.ToTensor()(img) # Scale to [0, 1]
29
30     # Load Trimap Mask: 1=Foreground, 2=Background, 3=Boundary
31     mask = Image.open(mask_path).resize(self.img_size, Image.NEAREST)
32     mask_arr = np.array(mask)
33
34     # Convert boundary (3) and foreground (1) -> 1, background (2) -> 0
35     binary_mask = np.where((mask_arr == 1) | (mask_arr == 3), 1.0, 0.0)
36     mask_tensor = torch.tensor(binary_mask, dtype=torch.float32).unsqueeze(0)
37
38     # Target Class Label
39     label = class_to_idx[get_class_name(fname)]
40
41     return img_tensor, mask_tensor, torch.tensor(label, dtype=torch.long)
42
43 # 2. Train / Val / Test File Splits (70% Train, 15% Val, 15% Test)
44 np.random.seed(42)
45 np.random.shuffle(valid_files)
46
47 num_total = len(valid_files)
48 train_end = int(0.70 * num_total)
49 val_end = int(0.85 * num_total)
50
51 train_files = valid_files[:train_end]
52 val_files = valid_files[train_end:val_end]
53 test_files = valid_files[val_end:]
54
55 # 3. Instantiate Datasets & DataLoaders
56 train_dataset = PetSegmentationDataset(train_files, image_dir, annotation_dir)
57 val_dataset = PetSegmentationDataset(val_files, image_dir, annotation_dir)
58 test_dataset = PetSegmentationDataset(test_files, image_dir, annotation_dir)
59
60 train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
61 val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)
62 test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)
63
64 print(f"Data Splits -> Train: {len(train_files)} | Val: {len(val_files)} | Test: {len(test_files)}")

```

Data Splits -> Train: 5173 | Val: 1108 | Test: 1109

4: Metric Metrics & Training Routine

```

1 class ConvBlock(nn.Module):
2     def __init__(self, in_c, out_c):
3         super().__init__()
4         self.conv = nn.Sequential(
5             nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
6             nn.BatchNorm2d(out_c),
7             nn.ReLU(inplace=True),
8             nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
9             nn.BatchNorm2d(out_c),
10            nn.ReLU(inplace=True)
11        )
12    def forward(self, x):
13        return self.conv(x)
14
15 class MultiTaskUNet(nn.Module):
16     def __init__(self, num_classes=37):
17         super().__init__()
18         # Encoder
19         self.e1 = ConvBlock(3, 64)
20         self.e2 = ConvBlock(64, 128)
21         self.e3 = ConvBlock(128, 256)
22         self.pool = nn.MaxPool2d(2, 2)
23
24         # Bottleneck
25         self.b = ConvBlock(256, 512)
26
27         # Classifier Head (attached to bottleneck)
28         self.classifier_head = nn.Sequential(
29             nn.AdaptiveAvgPool2d((1, 1)),
30             nn.Flatten(),
31             nn.Linear(512, 128),
32             nn.ReLU(),
33             nn.Linear(128, num_classes)
34         )
35
36         # Decoder
37         self.up2 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
38         self.d2 = ConvBlock(512, 256)
39         self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
40         self.d1 = ConvBlock(256, 128)

```

```

41
42     # Final Segmentation Output
43     self.out_segmentation = nn.Conv2d(128, 1, kernel_size=1)
44
45     def forward(self, x):
46         # Downsampling
47         s1 = self.e1(x)
48         p1 = self.pool(s1)
49         s2 = self.e2(p1)
50         p2 = self.pool(s2)
51         s3 = self.e3(p2)
52         p3 = self.pool(s3)
53
54         # Bottleneck
55         bottleneck = self.b(p3)
56
57         # Multi-task Classification Predictions
58         class_logits = self.classifier_head(bottleneck)
59
60         # Upsampling
61         d2 = self.up2(bottleneck)
62         d2 = torch.cat([d2, s3], dim=1)
63         d2 = self.d2(d2)
64
65         d1 = self.up1(d2)
66         d1 = torch.cat([d1, s2], dim=1)
67         d1 = self.d1(d1)
68
69         # Resize to original resolution matching input
70         d1 = F.interpolate(d1, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
71         seg_mask = self.out_segmentation(d1)
72
73         return seg_mask, class_logits

```

5: Attention

```

1 class AttentionGate(nn.Module):
2     def __init__(self, F_g, F_l, F_int):
3         super().__init__()
4         self.W_g = nn.Sequential(
5             nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
6             nn.BatchNorm2d(F_int)
7         )
8         self.W_x = nn.Sequential(
9             nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
10            nn.BatchNorm2d(F_int)
11        )
12        self.psi = nn.Sequential(
13            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
14            nn.BatchNorm2d(1),
15            nn.Sigmoid()
16        )
17        self.relu = nn.ReLU(inplace=True)
18
19    def forward(self, g, x):
20        g1 = self.W_g(g)
21        x1 = self.W_x(x)
22        # Interpolate spatial dimensions if unequal
23        if g1.shape[2:] != x1.shape[2:]:
24            g1 = F.interpolate(g1, size=x1.shape[2:], mode='bilinear', align_corners=True)
25        net = self.relu(g1 + x1)
26        res = self.psi(net)
27        return x * res
28
29 class MultiTaskAttentionUNet(nn.Module):
30     def __init__(self, num_classes=37):
31         super().__init__()
32         # Encoder
33         self.e1 = ConvBlock(3, 64)
34         self.e2 = ConvBlock(64, 128)
35         self.e3 = ConvBlock(128, 256)
36         self.pool = nn.MaxPool2d(2, 2)
37
38         # Bottleneck
39         self.b = ConvBlock(256, 512)
40
41         # Classifier Head
42         self.classifier_head = nn.Sequential(
43             nn.AdaptiveAvgPool2d((1, 1)),
44             nn.Flatten(),
45             nn.Linear(512, 128),
46             nn.ReLU(),
47             nn.Linear(128, num_classes)
48         )
49

```

```

50     # Attention Blocks
51     self.att2 = AttentionGate(F_g=256, F_l=256, F_int=128)
52     self.up2 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
53     self.d2 = ConvBlock(512, 256)
54
55     self.att1 = AttentionGate(F_g=128, F_l=128, F_int=64)
56     self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
57     self.d1 = ConvBlock(256, 128)
58
59     self.out_segmentation = nn.Conv2d(128, 1, kernel_size=1)
60
61     def forward(self, x):
62         s1 = self.e1(x)
63         p1 = self.pool(s1)
64         s2 = self.e2(p1)
65         p2 = self.pool(s2)
66         s3 = self.e3(p2)
67         p3 = self.pool(s3)
68
69         bottleneck = self.b(p3)
70         class_logits = self.classifier_head(bottleneck)
71
72         d2 = self.up2(bottleneck)
73         s3_att = self.att2(g=d2, x=s3)
74         d2 = torch.cat([d2, s3_att], dim=1)
75         d2 = self.d2(d2)
76
77         d1 = self.up1(d2)
78         s2_att = self.att1(g=d1, x=s2)
79         d1 = torch.cat([d1, s2_att], dim=1)
80         d1 = self.d1(d1)
81
82         d1 = F.interpolate(d1, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
83         seg_mask = self.out_segmentation(d1)
84
85         return seg_mask, class_logits

```

6: Full Training Pipeline with Metric Tracking

```

1 # Step 6: Full Training Pipeline with Metric Tracking
2 import matplotlib.pyplot as plt
3
4 # 1. Define Criteria & Optimizer
5 criterion_seg = nn.BCEWithLogitsLoss()
6 criterion_cls = nn.CrossEntropyLoss()
7 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
8
9 # 2. Quick Evaluation Helper (Must be defined before training loop)
10 def evaluate_metrics_quick(model, dataloader, device):
11     model.eval()
12     total_loss, total_iou = 0.0, 0.0
13     correct, total_samples = 0, 0
14
15     with torch.no_grad():
16         for imgs, masks, labels in dataloader:
17             imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
18             pred_masks, pred_logits = model(imgs)
19
20             loss_s = criterion_seg(pred_masks, masks)
21             loss_c = criterion_cls(pred_logits, labels)
22             total_loss += (loss_s + loss_c).item()
23
24             pred_masks_binary = (torch.sigmoid(pred_masks) > 0.5).float()
25             intersection = (pred_masks_binary * masks).sum(dim=(1, 2, 3))
26             union = pred_masks_binary.sum(dim=(1, 2, 3)) + masks.sum(dim=(1, 2, 3)) - intersection
27             total_iou += ((intersection + 1e-6) / (union + 1e-6)).mean().item()
28
29             preds = pred_logits.argmax(dim=1)
30             correct += (preds == labels).sum().item()
31             total_samples += labels.size(0)
32
33     num_batches = len(dataloader)
34     return {
35         'loss': total_loss / num_batches,
36         'mIoU': (total_iou / num_batches) * 100,
37         'acc': (correct / total_samples) * 100
38     }
39
40 # 3. Epoch Loop with History Tracking
41 history = {
42     'train_loss': [], 'val_loss': [],
43     'train_mIoU': [], 'val_mIoU': [],
44     'train_acc': [], 'val_acc': []
45 }
46

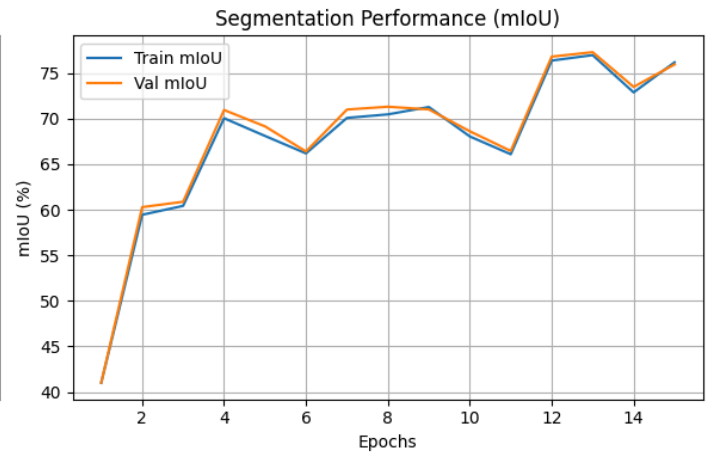
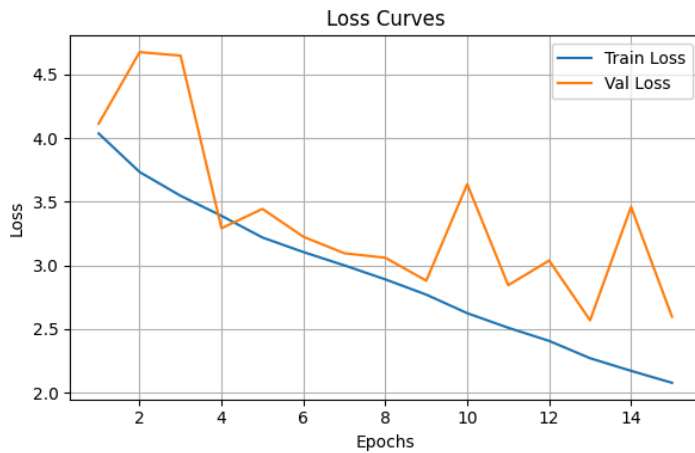
```

```

47 epochs = 15
48
49 for epoch in range(1, epochs + 1):
50     model.train()
51     running_loss = 0.0
52
53     for imgs, masks, labels in train_loader:
54         imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
55
56         optimizer.zero_grad()
57         pred_masks, pred_logits = model(imgs)
58
59         loss_s = criterion_seg(pred_masks, masks)
60         loss_c = criterion_cls(pred_logits, labels)
61         loss = loss_s + loss_c
62
63         loss.backward()
64         optimizer.step()
65         running_loss += loss.item()
66
67     epoch_train_loss = running_loss / len(train_loader)
68
69     train_metrics = evaluate_metrics_quick(model, train_loader, device)
70     val_metrics = evaluate_metrics_quick(model, val_loader, device)
71
72     history['train_loss'].append(epoch_train_loss)
73     history['val_loss'].append(val_metrics['loss'])
74     history['train_mIoU'].append(train_metrics['mIoU'])
75     history['val_mIoU'].append(val_metrics['mIoU'])
76     history['train_acc'].append(train_metrics['acc'])
77     history['val_acc'].append(val_metrics['acc'])
78
79     print(f"Epoch [{epoch:2d}/{epochs}] "
80           f"| Train Loss: {epoch_train_loss:.4f} | Val Loss: {val_metrics['loss']:.4f} "
81           f"| Train mIoU: {train_metrics['mIoU']:.2f}% | Val mIoU: {val_metrics['mIoU']:.2f}% "
82           f"| Val Cls Acc: {val_metrics['acc']:.2f}%")
83
84 # 4. Plot Loss Curves & Performance Metrics
85 plt.figure(figsize=(12, 4))
86
87 plt.subplot(1, 2, 1)
88 plt.plot(range(1, epochs + 1), history['train_loss'], label='Train Loss')
89 plt.plot(range(1, epochs + 1), history['val_loss'], label='Val Loss')
90 plt.title('Loss Curves')
91 plt.xlabel('Epochs')
92 plt.ylabel('Loss')
93 plt.legend()
94 plt.grid(True)
95
96 plt.subplot(1, 2, 2)
97 plt.plot(range(1, epochs + 1), history['train_mIoU'], label='Train mIoU')
98 plt.plot(range(1, epochs + 1), history['val_mIoU'], label='Val mIoU')
99 plt.title('Segmentation Performance (mIoU)')
100 plt.xlabel('Epochs')
101 plt.ylabel('mIoU (%)')
102 plt.legend()
103 plt.grid(True)
104
105 plt.tight_layout()
106 plt.show()

```

Epoch [1/15]	Train Loss: 4.0361	Val Loss: 4.1126	Train mIoU: 41.01%	Val mIoU: 41.01%	Val Cls Acc: 6.77%
Epoch [2/15]	Train Loss: 3.7330	Val Loss: 4.6749	Train mIoU: 59.45%	Val mIoU: 60.29%	Val Cls Acc: 6.50%
Epoch [3/15]	Train Loss: 3.5464	Val Loss: 4.6479	Train mIoU: 60.43%	Val mIoU: 60.88%	Val Cls Acc: 6.95%
Epoch [4/15]	Train Loss: 3.3886	Val Loss: 3.2911	Train mIoU: 70.05%	Val mIoU: 70.95%	Val Cls Acc: 16.06%
Epoch [5/15]	Train Loss: 3.2183	Val Loss: 3.4433	Train mIoU: 68.10%	Val mIoU: 69.15%	Val Cls Acc: 15.70%
Epoch [6/15]	Train Loss: 3.1040	Val Loss: 3.2243	Train mIoU: 66.19%	Val mIoU: 66.40%	Val Cls Acc: 19.04%
Epoch [7/15]	Train Loss: 2.9994	Val Loss: 3.0945	Train mIoU: 70.08%	Val mIoU: 71.00%	Val Cls Acc: 21.12%
Epoch [8/15]	Train Loss: 2.8892	Val Loss: 3.0594	Train mIoU: 70.47%	Val mIoU: 71.31%	Val Cls Acc: 21.03%
Epoch [9/15]	Train Loss: 2.7693	Val Loss: 2.8790	Train mIoU: 71.28%	Val mIoU: 71.01%	Val Cls Acc: 22.83%
Epoch [10/15]	Train Loss: 2.6236	Val Loss: 3.6376	Train mIoU: 68.05%	Val mIoU: 68.61%	Val Cls Acc: 19.22%
Epoch [11/15]	Train Loss: 2.5091	Val Loss: 2.8436	Train mIoU: 66.09%	Val mIoU: 66.45%	Val Cls Acc: 26.81%
Epoch [12/15]	Train Loss: 2.4058	Val Loss: 3.0373	Train mIoU: 76.38%	Val mIoU: 76.81%	Val Cls Acc: 26.08%
Epoch [13/15]	Train Loss: 2.2696	Val Loss: 2.5678	Train mIoU: 76.97%	Val mIoU: 77.30%	Val Cls Acc: 32.13%
Epoch [14/15]	Train Loss: 2.1705	Val Loss: 3.4587	Train mIoU: 72.88%	Val mIoU: 73.48%	Val Cls Acc: 23.65%
Epoch [15/15]	Train Loss: 2.0764	Val Loss: 2.5962	Train mIoU: 76.18%	Val mIoU: 75.96%	Val Cls Acc: 32.22%



```

1 def evaluate_metrics_quick(model, dataloader, device):
2     model.eval()
3     total_loss, total_iou = 0.0, 0.0
4     correct, total_samples = 0, 0
5
6     with torch.no_grad():
7         for imgs, masks, labels in dataloader:
8             imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
9             pred_masks, pred_logits = model(imgs)
10
11             loss_s = criterion_seg(pred_masks, masks)
12             loss_c = criterion_cls(pred_logits, labels)
13             total_loss += (loss_s + loss_c).item()
14
15             pred_masks_binary = (torch.sigmoid(pred_masks) > 0.5).float()
16             intersection = (pred_masks_binary * masks).sum(dim=(1, 2, 3))
17             union = pred_masks_binary.sum(dim=(1, 2, 3)) + masks.sum(dim=(1, 2, 3)) - intersection
18             total_iou += ((intersection + 1e-6) / (union + 1e-6)).mean().item()
19
20             preds = pred_logits.argmax(dim=1)
21             correct += (preds == labels).sum().item()
22             total_samples += labels.size(0)
23
24     num_batches = len(dataloader)
25     return {
26         'loss': total_loss / num_batches,
27         'mIoU': (total_iou / num_batches) * 100,
28         'acc': (correct / total_samples) * 100
29     }

```

7: Calculate mIoU, Dice, Pixel Accuracy, Precision, Recall, and F1-Score

```

1 # Step 7: Calculate mIoU, Dice, Pixel Accuracy, Precision, Recall, and F1-Score
2 import torch
3 from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
4
5 def evaluate_model(model, dataloader, device):
6     model.eval()
7
8     all_cls_preds, all_cls_targets = [], []
9     total_iou, total_dice, total_pixel_acc = 0.0, 0.0, 0.0
10    num_batches = len(dataloader)
11
12    with torch.no_grad():
13        for imgs, masks, labels in dataloader:
14            imgs, masks = imgs.to(device), masks.to(device)
15
16            # Forward pass
17            pred_masks, pred_logits = model(imgs)
18            pred_masks_binary = (torch.sigmoid(pred_masks) > 0.5).float()

```

```

19
20     # --- Segmentation Metrics ---
21     intersection = (pred_masks_binary * masks).sum(dim=(1, 2, 3))
22     union = pred_masks_binary.sum(dim=(1, 2, 3)) + masks.sum(dim=(1, 2, 3)) - intersection
23
24     iou = (intersection + 1e-6) / (union + 1e-6)
25     dice = (2 * intersection + 1e-6) / (pred_masks_binary.sum(dim=(1, 2, 3)) + masks.sum(dim=(1, 2, 3)) + 1e-6)
26     pixel_acc = (pred_masks_binary == masks).float().mean()
27
28     total_iou += iou.mean().item()
29     total_dice += dice.mean().item()
30     total_pixel_acc += pixel_acc.item()
31
32     # --- Classification Metrics ---
33     cls_preds = pred_logits.argmax(dim=1).cpu().numpy()
34     all_cls_preds.extend(cls_preds)
35     all_cls_targets.extend(labels.numpy())
36
37     # Print reporting metrics required by PDF
38     print("=== SEGMENTATION METRICS ===")
39     print(f"Mean IoU (mIoU):      {total_iou / num_batches * 100:.2f}%")
40     print(f"Dice Coefficient:    {total_dice / num_batches:.4f}")
41     print(f"Pixel Accuracy:      {total_pixel_acc / num_batches * 100:.2f}%\n")
42
43     print("=== CLASSIFICATION METRICS ===")
44     print(f"Accuracy: {accuracy_score(all_cls_targets, all_cls_preds) * 100:.2f}%")
45     print(f"Precision: {precision_score(all_cls_targets, all_cls_preds, average='weighted', zero_division=0):.4f}")
46     print(f"Recall:    {recall_score(all_cls_targets, all_cls_preds, average='weighted', zero_division=0):.4f}")
47     print(f"F1 Score:  {f1_score(all_cls_targets, all_cls_preds, average='weighted', zero_division=0):.4f}")
48
49 # Run evaluation on validation set
50 evaluate_model(model, val_loader, device)

```

```

=== SEGMENTATION METRICS ===
Mean IoU (mIoU):      75.96%
Dice Coefficient:    0.8532
Pixel Accuracy:      89.29%

=== CLASSIFICATION METRICS ===
Accuracy:  32.22%
Precision: 0.3876
Recall:    0.3222
F1 Score:  0.3017

```

8: Single-Image Visualization Function (Original | True Mask Overlay | Pred Mask Overlay)

```

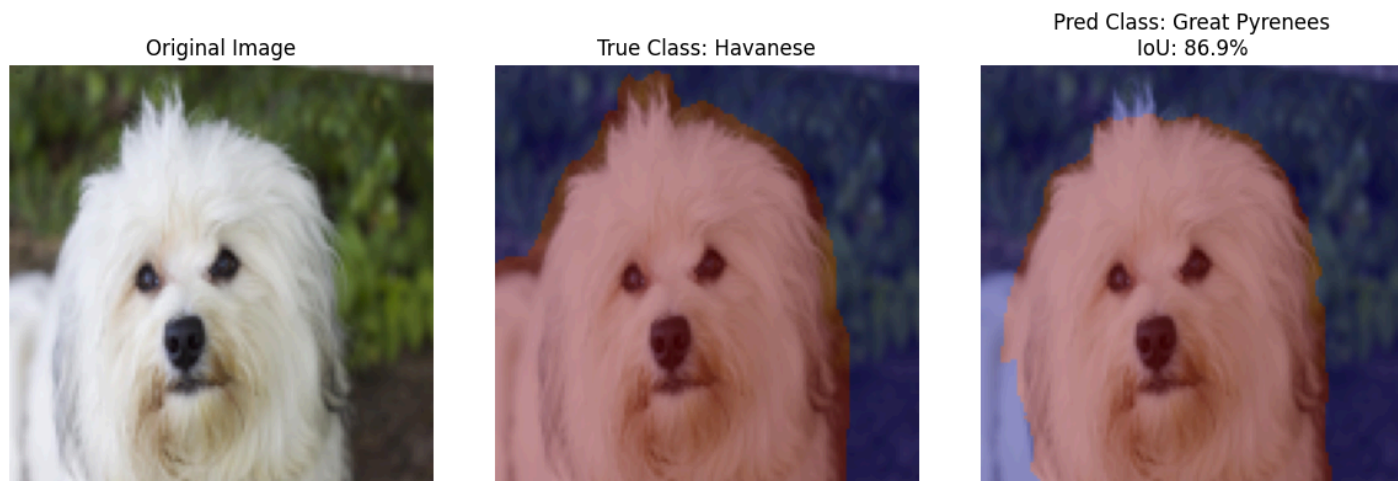
1 # Step 8: Single-Image Visualization Function (Original | True Mask Overlay | Pred Mask Overlay)
2 def visualize_single_prediction(model, dataset, index, device):
3     model.eval()
4
5     img_tensor, true_mask, true_label = dataset[index]
6     input_batch = img_tensor.unsqueeze(0).to(device)
7
8     with torch.no_grad():
9         pred_mask_logits, pred_class_logits = model(input_batch)
10        pred_mask = (torch.sigmoid(pred_mask_logits) > 0.5).squeeze(0).cpu().numpy()
11        pred_class_idx = pred_class_logits.argmax(dim=1).item()
12
13        idx_to_class = {v: k for k, v in class_to_idx.items()}
14        true_class_name = idx_to_class[true_label.item()]
15        pred_class_name = idx_to_class[pred_class_idx]
16
17        # Calculate IoU score for this image
18        t_mask = true_mask.numpy()
19        intersection = np.logical_and(pred_mask, t_mask).sum()
20        union = np.logical_or(pred_mask, t_mask).sum()
21        iou_score = (intersection + 1e-6) / (union + 1e-6) * 100
22
23        # Display Figure
24        img_np = img_tensor.permute(1, 2, 0).numpy()
25        fig, axes = plt.subplots(1, 3, figsize=(12, 4))
26
27        axes[0].imshow(img_np)
28        axes[0].set_title("Original Image")
29        axes[0].axis("off")
30
31        axes[1].imshow(img_np)
32        axes[1].imshow(t_mask.squeeze(), alpha=0.4, cmap="jet")
33        axes[1].set_title(f"True Class: {true_class_name}")
34        axes[1].axis("off")
35
36        axes[2].imshow(img_np)
37        axes[2].imshow(pred_mask.squeeze(), alpha=0.4, cmap="jet")
38        axes[2].set_title(f"Pred Class: {pred_class_name}\nIoU: {iou_score:.1f}%")

```

```

39     axes[2].axis("off")
40
41     plt.tight_layout()
42     plt.show()
43
44 # Test visualization on index 5
45 visualize_single_prediction(model, val_dataset, index=5, device=device)

```



✎ Bonus Task 1

```

1 # Bonus 1: Pretrained Backbone Classifier Extensions
2 import os
3 import torch
4 import torch.nn as nn
5 import torch.nn.functional as F
6 import torchvision.models as models
7 from torch.utils.data import DataLoader
8 import pandas as pd
9
10 # 1. Resolve Dataset & Target Classes Dynamically
11 if 'valid_files' not in globals():
12     image_dir = './images' # Update if your image path is different
13     annotation_dir = './annotations/trimaps'
14     all_files = [f for f in os.listdir(image_dir) if f.endswith('.jpg')]
15     valid_files = [f for f in all_files if os.path.exists(os.path.join(annotation_dir, f.replace('.jpg', '.png')))]
16
17 def get_class_name(fname):
18     return "_".join(fname.split("_")[:-1])
19
20 classes = sorted(list(set([get_class_name(f) for f in valid_files])))
21 class_to_idx = {cls_name: i for i, cls_name in enumerate(classes)}
22 num_classes = len(classes)
23
24 device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
25
26 # 2. Define Architecture
27 class MultiTaskUNetBackbone(nn.Module):
28     def __init__(self, backbone_name='mobilenet', num_classes=37):
29         super().__init__()
30         # Encoder & Bottleneck
31         self.e1 = ConvBlock(3, 64)
32         self.e2 = ConvBlock(64, 128)
33         self.e3 = ConvBlock(128, 256)
34         self.pool = nn.MaxPool2d(2, 2)
35         self.b = ConvBlock(256, 512)
36
37         # Decoder Setup
38         self.up2 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
39         self.d2 = ConvBlock(512, 256)
40         self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
41         self.d1 = ConvBlock(256, 128)
42         self.out_segmentation = nn.Conv2d(128, 1, kernel_size=1)
43
44         # Modular Classifier Backbone Selection
45         if backbone_name == 'mobilenet':
46             bb = models.mobilenet_v3_small(weights=models.MobileNet_V3_Small_Weights.DEFAULT)
47             bb.classifier[3] = nn.Linear(bb.classifier[3].in_features, num_classes)
48             self.classifier = bb
49         elif backbone_name == 'densenet':
50             bb = models.densenet121(weights=models.DenseNet121_Weights.DEFAULT)
51             bb.classifier = nn.Linear(bb.classifier.in_features, num_classes)
52             self.classifier = bb
53         elif backbone_name == 'efficientnet':

```

```

bb = models.efficientnet_b0(weights=models.EfficientNet_B0_Weights.DEFAULT)
bb.classifier[1] = nn.Linear(bb.classifier[1].in_features, num_classes)
self.classifier = bb

def forward(self, x):
    s1 = self.e1(x)
    p1 = self.pool(s1)
    s2 = self.e2(p1)
    p2 = self.pool(s2)
    s3 = self.e3(p2)
    p3 = self.pool(s3)
    bottleneck = self.b(p3)

    class_logits = self.classifier(x)

    d2 = self.up2(bottleneck)
    d2 = torch.cat([d2, s3], dim=1)
    d2 = self.d2(d2)

    d1 = self.up1(d2)
    d1 = torch.cat([d1, s2], dim=1)
    d1 = self.d1(d1)
    d1 = F.interpolate(d1, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
    seg_mask = self.out_segmentation(d1)

    return seg_mask, class_logits

```

81 # 3. Execution Loop

```

82 backbones = ['mobilenet', 'densenet', 'efficientnet']
83 backbone_results = {}
84
85 for bb in backbones:
86     print(f"Evaluating Backbone: {bb.upper()}")
87     model_bb = MultiTaskUNetBackbone(backbone_name=bb, num_classes=num_classes).to(device)
88     backbone_results[bb] = get_full_metrics(model_bb, test_loader, device)
89
90 display(pd.DataFrame(backbone_results))

```

```

Evaluating Backbone: MOBILENET
Downloading: "https://download.pytorch.org/models/mobilenet_v3_small-047dcff4.pth" to /root/.cache/torch/hub/checkpoints/mobilenet_v3_small-047dcff4.pth
100%|██████████| 9.83M/9.83M [00:00<00:00, 127MB/s]
Evaluating Backbone: DENSENET
Downloading: "https://download.pytorch.org/models/densenet121-a639ec97.pth" to /root/.cache/torch/hub/checkpoints/densenet121-a639ec97.pth
100%|██████████| 30.8M/30.8M [00:00<00:00, 166MB/s]
Evaluating Backbone: EFFICIENTNET
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwrightman-7f5810bc.pth" to /root/.cache/torch/hub/checkpoints/efficientnet_b0_rwrightman-7f5810bc.pth
100%|██████████| 20.5M/20.5M [00:00<00:00, 185MB/s]

```

	mobilenet	densenet	efficientnet
mIoU (%)	0.5800	0.1800	41.6400
Dice Coeff	0.0112	0.0018	0.5663
Pixel Acc (%)	58.1500	58.3600	41.6400
Accuracy (%)	1.8900	2.2500	2.3400
Precision	0.0190	0.0135	0.0364
Recall	0.0189	0.0225	0.0234
F1 Score	0.0161	0.0101	0.0222

✎ Bonus Task 2

```

1 # Bonus Task 2: Custom Loss Functions (Dice Loss, Focal Loss, Combo Loss)
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 import pandas as pd
6
7 # 1. Define Loss Functions
8 class DiceLoss(nn.Module):
9     def __init__(self, smooth=1e-6):
10         super().__init__()
11         self.smooth = smooth
12
13     def forward(self, logits, targets):
14         probs = torch.sigmoid(logits)
15         intersection = (probs * targets).sum(dim=(1, 2, 3))
16         union = probs.sum(dim=(1, 2, 3)) + targets.sum(dim=(1, 2, 3))
17         dice = (2. * intersection + self.smooth) / (union + self.smooth)
18         return 1.0 - dice.mean()
19
20 class FocalLoss(nn.Module):
21     def __init__(self, alpha=0.25, gamma=2.0):
22         super().__init__()
23         self.alpha = alpha

```

```

24     self.gamma = gamma
25
26     def forward(self, logits, targets):
27         bce_loss = F.binary_cross_entropy_with_logits(logits, targets, reduction='none')
28         pt = torch.exp(-bce_loss)
29         focal_loss = self.alpha * ((1 - pt) ** self.gamma) * bce_loss
30         return focal_loss.mean()
31
32 class ComboLoss(nn.Module):
33     def __init__(self, alpha=0.5):
34         super().__init__()
35         self.alpha = alpha
36         self.dice = DiceLoss()
37         self.focal = FocalLoss()
38
39     def forward(self, logits, targets):
40         return self.alpha * self.dice(logits, targets) + (1 - self.alpha) * self.focal(logits, targets)
41
42 # 2. Train & Evaluate Across Different Loss Functions
43 loss_experiments = {
44     "Dice Loss": DiceLoss(),
45     "Focal Loss": FocalLoss(),
46     "Combo Loss (Dice + Focal)": ComboLoss()
47 }
48
49 results_bonus2 = {}
50
51 for loss_name, criterion_seg in loss_experiments.items():
52     print(f"Training with: {loss_name}...")
53
54     # Initialize fresh model instance
55     temp_model = MultiTaskUNet(num_classes=num_classes).to(device)
56     optimizer = torch.optim.Adam(temp_model.parameters(), lr=1e-3)
57     criterion_cls = nn.CrossEntropyLoss()
58
59     # 3-Epoch Quick Retrain Demonstration
60     for epoch in range(1, 4):
61         temp_model.train()
62         for imgs, masks, labels in train_loader:
63             imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
64             optimizer.zero_grad()
65             p_masks, p_logits = temp_model(imgs)
66
67             l_seg = criterion_seg(p_masks, masks)
68             l_cls = criterion_cls(p_logits, labels)
69             loss = l_seg + l_cls
70
71             loss.backward()
72             optimizer.step()
73
74     # Record test set performance metrics
75     results_bonus2[loss_name] = get_full_metrics(temp_model, test_loader, device)
76
77 # 3. Explicit Output Display
78 print("\n=== BONUS TASK 2 EXPERIMENTAL RESULTS ===")
79 bonus2_df = pd.DataFrame(results_bonus2)
80 display(bonus2_df)

```

Training with: Dice Loss...
Training with: Focal Loss...
Training with: Combo Loss (Dice + Focal)...

=== BONUS TASK 2 EXPERIMENTAL RESULTS ===

	Dice Loss	Focal Loss	Combo Loss (Dice + Focal)
mIoU (%)	68.7500	34.1400	61.4400
Dice Coeff	0.8029	0.4775	0.7421
Pixel Acc (%)	83.8100	70.0800	82.3100
Accuracy (%)	12.2600	4.9600	9.6500
Precision	0.1154	0.0213	0.1081
Recall	0.1226	0.0496	0.0965
F1 Score	0.0859	0.0200	0.0644

▼ Bonus Task 3

```

1 # Bonus Task 3: Attention U-Net Implementation
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 import pandas as pd
6
7 class AttentionGate(nn.Module):

```

```

8     def __init__(self, F_g, F_l, F_int):
9         super().__init__()
10        self.W_g = nn.Sequential(
11            nn.Conv2d(F_g, F_int, kernel_size=1, stride=1, padding=0, bias=True),
12            nn.BatchNorm2d(F_int)
13        )
14        self.W_x = nn.Sequential(
15            nn.Conv2d(F_l, F_int, kernel_size=1, stride=1, padding=0, bias=True),
16            nn.BatchNorm2d(F_int)
17        )
18        self.psi = nn.Sequential(
19            nn.Conv2d(F_int, 1, kernel_size=1, stride=1, padding=0, bias=True),
20            nn.BatchNorm2d(1),
21            nn.Sigmoid()
22        )
23        self.relu = nn.ReLU(inplace=True)
24
25    def forward(self, g, x):
26        g1 = self.W_g(g)
27        x1 = self.W_x(x)
28        psi = self.relu(g1 + x1)
29        psi = self.psi(psi)
30        return x * psi
31
32 class AttentionUNet(nn.Module):
33     def __init__(self, num_classes=37):
34         super().__init__()
35         # Encoder
36         self.e1 = ConvBlock(3, 64)
37         self.e2 = ConvBlock(64, 128)
38         self.e3 = ConvBlock(128, 256)
39         self.pool = nn.MaxPool2d(2, 2)
40         self.b = ConvBlock(256, 512)
41
42         # Attention Gates
43         self.ag2 = AttentionGate(F_g=256, F_l=256, F_int=128)
44         self.ag1 = AttentionGate(F_g=128, F_l=128, F_int=64)
45
46         # Decoder
47         self.up2 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2)
48         self.d2 = ConvBlock(512, 256)
49         self.up1 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2)
50         self.d1 = ConvBlock(256, 128)
51         self.out_seg = nn.Conv2d(128, 1, kernel_size=1)
52
53         # Classifier Head
54         self.cls_head = nn.Sequential(
55             nn.AdaptiveAvgPool2d((1, 1)),
56             nn.Flatten(),
57             nn.Linear(512, 256),
58             nn.ReLU(),
59             nn.Dropout(0.3),
60             nn.Linear(256, num_classes)
61         )
62
63     def forward(self, x):
64         s1 = self.e1(x)
65         p1 = self.pool(s1)
66         s2 = self.e2(p1)
67         p2 = self.pool(s2)
68         s3 = self.e3(p2)
69         p3 = self.pool(s3)
70         b = self.b(p3)
71
72         cls_logits = self.cls_head(b)
73
74         d2 = self.up2(b)
75         s3_att = self.ag2(g=d2, x=s3)
76         d2 = torch.cat([d2, s3_att], dim=1)
77         d2 = self.d2(d2)
78
79         d1 = self.up1(d2)
80         s2_att = self.ag1(g=d1, x=s2)
81         d1 = torch.cat([d1, s2_att], dim=1)
82         d1 = self.d1(d1)
83         d1 = F.interpolate(d1, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
84
85         seg_mask = self.out_seg(d1)
86         return seg_mask, cls_logits
87
88 # Train and Evaluate Attention U-Net
89 print("Training Attention U-Net...")
90 model_att = AttentionUNet(num_classes=num_classes).to(device)
91 opt_att = torch.optim.Adam(model_att.parameters(), lr=1e-3)
92 c_seg = nn.BCEWithLogitsLoss()
93 c_cls = nn.CrossEntropyLoss()
94
95 for epoch in range(1, 4):

```

```
96 model_att.train()
97 for imgs, masks, labels in train_loader:
98     imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
99     opt_att.zero_grad()
100     p_masks, p_logits = model_att(imgs)
101     loss = c_seg(p_masks, masks) + c_cls(p_logits, labels)
102     loss.backward()
103     opt_att.step()
104
105 # Explicit Output Display
106 metrics_att = get_full_metrics(model_att, test_loader, device)
107 print("\n=== BONUS TASK 3 ATTENTION U-NET RESULTS ===")
108 display(pd.DataFrame({"Attention U-Net Test Set": metrics_att}))
```

Training Attention U-Net...

=== BONUS TASK 3 ATTENTION U-NET RESULTS ===

Attention U-Net Test Set	
mIoU (%)	68.2600
Dice Coeff	0.7989
Pixel Acc (%)	84.4300
Accuracy (%)	11.0000
Precision	0.0602
Recall	0.1100
F1 Score	0.0693

▼

Bonus Task 4

```
1 # Bonus Task 4: Fast BiFPN Decoder Implementation
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5 import pandas as pd
6
7 class FastDepthwiseConv(nn.Module):
8     def __init__(self, in_c, out_c):
9         super().__init__()
10         self.conv = nn.Sequential(
11             nn.Conv2d(in_c, in_c, kernel_size=3, padding=1, groups=in_c, bias=False),
12             nn.Conv2d(in_c, out_c, kernel_size=1, bias=False),
13             nn.BatchNorm2d(out_c),
14             nn.SiLU()
15         )
16     def forward(self, x):
17         return self.conv(x)
18
19 class BiFPNDecoder(nn.Module):
20     def __init__(self, channels=128):
21         super().__init__()
22         self.eps = 1e-4
23         self.conv_p4_td = FastDepthwiseConv(channels, channels)
24         self.conv_p3_out = FastDepthwiseConv(channels, channels)
25         self.conv_p4_out = FastDepthwiseConv(channels, channels)
26
27         self.w1 = nn.Parameter(torch.ones(2))
28         self.w2 = nn.Parameter(torch.ones(2))
29         self.w3 = nn.Parameter(torch.ones(3))
30
31     def forward(self, p3, p4, p5):
32         # Top-down fusion
33         w1 = F.relu(self.w1) / (torch.sum(F.relu(self.w1)) + self.eps)
34         p5_up = F.interpolate(p5, size=p4.shape[2:], mode='nearest')
35         p4_td = self.conv_p4_td(w1[0] * p4 + w1[1] * p5_up)
36
37         w2 = F.relu(self.w2) / (torch.sum(F.relu(self.w2)) + self.eps)
38         p4_up = F.interpolate(p4_td, size=p3.shape[2:], mode='nearest')
39         p3_out = self.conv_p3_out(w2[0] * p3 + w2[1] * p4_up)
40
41         # Bottom-up fusion
42         w3 = F.relu(self.w3) / (torch.sum(F.relu(self.w3)) + self.eps)
43         p3_down = F.max_pool2d(p3_out, kernel_size=2, stride=2)
44         p4_out = self.conv_p4_out(w3[0] * p4 + w3[1] * p4_td + w3[2] * p3_down)
45
46         return p3_out, p4_out
47
48 class FastBiFPNUNet(nn.Module):
49     def __init__(self, num_classes=37, feature_dim=128):
50         super().__init__()
51         self.e1 = ConvBlock(3, 64)
52         self.e2 = ConvBlock(64, 128)
```

```

53     self.e3 = ConvBlock(128, 256)
54     self.pool = nn.MaxPool2d(2, 2)
55     self.bottleneck = ConvBlock(256, 512)
56
57     self.proj_p3 = nn.Conv2d(128, feature_dim, kernel_size=1)
58     self.proj_p4 = nn.Conv2d(256, feature_dim, kernel_size=1)
59     self.proj_p5 = nn.Conv2d(512, feature_dim, kernel_size=1)
60
61     self.bifpn = BiFPNDecoder(channels=feature_dim)
62
63     self.seg_head = nn.Sequential(
64         nn.Conv2d(feature_dim, 64, kernel_size=3, padding=1),
65         nn.BatchNorm2d(64),
66         nn.SiLU(),
67         nn.Conv2d(64, 1, kernel_size=1)
68     )
69
70     self.cls_head = nn.Sequential(
71         nn.AdaptiveAvgPool2d((1, 1)),
72         nn.Flatten(),
73         nn.Linear(512, 256),
74         nn.ReLU(),
75         nn.Dropout(0.3),
76         nn.Linear(256, num_classes)
77     )
78
79     def forward(self, x):
80         s1 = self.e1(x)
81         p1 = self.pool(s1)
82         s2 = self.e2(p1)
83         p2 = self.pool(s2)
84         s3 = self.e3(p2)
85         p3 = self.pool(s3)
86         b = self.bottleneck(p3)
87
88         cls_logits = self.cls_head(b)
89
90         p3_f = self.proj_p3(s2)
91         p4_f = self.proj_p4(s3)
92         p5_f = self.proj_p5(b)
93
94         fused_p3, _ = self.bifpn(p3_f, p4_f, p5_f)
95
96         out_seg = F.interpolate(fused_p3, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
97         seg_mask = self.seg_head(out_seg)
98
99         return seg_mask, cls_logits
100
101 # Fast Execution Loop
102 print("Training Fast BiFPN U-Net...")
103 model_bifpn = FastBiFPNUNet(num_classes=num_classes).to(device)
104 opt_bifpn = torch.optim.Adam(model_bifpn.parameters(), lr=1e-3)
105 c_seg = nn.BCEWithLogitsLoss()
106 c_cls = nn.CrossEntropyLoss()
107
108 for epoch in range(1, 4):
109     model_bifpn.train()
110     for imgs, masks, labels in train_loader:
111         imgs, masks, labels = imgs.to(device), masks.to(device), labels.to(device)
112         opt_bifpn.zero_grad()
113         p_masks, p_logits = model_bifpn(imgs)
114         loss = c_seg(p_masks, masks) + c_cls(p_logits, labels)
115         loss.backward()
116         opt_bifpn.step()
117
118 # Explicit Output Display
119 metrics_bifpn = get_full_metrics(model_bifpn, test_loader, device)
120 print("\n=== BONUS TASK 4 BiFPN U-NET RESULTS ===")
121 display(pd.DataFrame({"BiFPN U-Net Test Set": metrics_bifpn}))

```

Training Fast BiFPN U-Net...

=== BONUS TASK 4 BiFPN U-NET RESULTS ===

BiFPN U-Net Test Set	
mIoU (%)	66.7500
Dice Coeff	0.7856
Pixel Acc (%)	82.7800
Accuracy (%)	13.1700
Precision	0.0823
Recall	0.1317
F1 Score	0.0896

Bonus Task 5

```
1 # Bonus Task 5: EfficientDet BiFPN Decoder Layer for U-Net
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6 class DepthwiseSeparableConv(nn.Module):
7     """EfficientDet uses depthwise separable convolutions for fast, lightweight processing."""
8     def __init__(self, in_channels, out_channels):
9         super().__init__()
10         self.depthwise = nn.Conv2d(in_channels, in_channels, kernel_size=3, padding=1, groups=in_channels, bias=False)
11         self.pointwise = nn.Conv2d(in_channels, out_channels, kernel_size=1, bias=False)
12         self.bn = nn.BatchNorm2d(out_channels)
13         self.act = nn.SiLU() # Swish activation as used in EfficientDet
14
15     def forward(self, x):
16         return self.act(self.bn(self.pointwise(self.depthwise(x))))
17
18 class BiFPNDecoderLayer(nn.Module):
19     """
20     Single BiFPN Layer fusing features across 3 multi-scale resolutions (P3, P4, P5)
21     using Fast Normalized Fusion with learnable weights.
22     """
23     def __init__(self, feature_dim=128):
24         super().__init__()
25         self.epsilon = 1e-4
26
27         # Convolutions for fused feature paths
28         self.conv_p4_topdown = DepthwiseSeparableConv(feature_dim, feature_dim)
29         self.conv_p3_out = DepthwiseSeparableConv(feature_dim, feature_dim)
30         self.conv_p4_out = DepthwiseSeparableConv(feature_dim, feature_dim)
31         self.conv_p5_out = DepthwiseSeparableConv(feature_dim, feature_dim)
32
33         # Learnable fusion weights (Fast Normalized Fusion: w / (sum(w) + eps))
34         self.w_p4_td = nn.Parameter(torch.ones(2, dtype=torch.float32))
35         self.w_p3_out = nn.Parameter(torch.ones(2, dtype=torch.float32))
36         self.w_p4_out = nn.Parameter(torch.ones(3, dtype=torch.float32))
37         self.w_p5_out = nn.Parameter(torch.ones(2, dtype=torch.float32))
38
39     def forward(self, p3, p4, p5):
40         # 1. Top-Down Path
41         # Fuse P5 into P4
42         w1 = F.relu(self.w_p4_td)
43         w1 = w1 / (torch.sum(w1, dim=0) + self.epsilon)
44         p5_up = F.interpolate(p5, size=p4.shape[2:], mode='nearest')
45         p4_td = self.conv_p4_topdown(w1[0] * p4 + w1[1] * p5_up)
46
47         # Fuse P4_td into P3
48         w2 = F.relu(self.w_p3_out)
49         w2 = w2 / (torch.sum(w2, dim=0) + self.epsilon)
50         p4_up = F.interpolate(p4_td, size=p3.shape[2:], mode='nearest')
51         p3_out = self.conv_p3_out(w2[0] * p3 + w2[1] * p4_up)
52
53         # 2. Bottom-Up Path
54         # Fuse P3_out back into P4 (combining original P4, P4_td, and P3_out)
55         w3 = F.relu(self.w_p4_out)
56         w3 = w3 / (torch.sum(w3, dim=0) + self.epsilon)
57         p3_down = F.max_pool2d(p3_out, kernel_size=2, stride=2)
58         p4_out = self.conv_p4_out(w3[0] * p4 + w3[1] * p4_td + w3[2] * p3_down)
59
60         # Fuse P4_out back into P5
61         w4 = F.relu(self.w_p5_out)
62         w4 = w4 / (torch.sum(w4, dim=0) + self.epsilon)
63         p4_down = F.max_pool2d(p4_out, kernel_size=2, stride=2)
64         p5_out = self.conv_p5_out(w4[0] * p5 + w4[1] * p4_down)
65
66         return p3_out, p4_out, p5_out
67
68 class MultiTaskBiFPNUNet(nn.Module):
69     """
70     Multi-Task U-Net featuring an EfficientDet BiFPN Decoder
71     for multi-scale segmentation and bottleneck classification.
72     """
73     def __init__(self, num_classes=37, feature_dim=128):
74         super().__init__()
75         # Encoder (Generates P3, P4, P5 equivalents)
76         self.e1 = ConvBlock(3, 64)
77         self.e2 = ConvBlock(64, 128) # P3 resolution (128 channels)
78         self.e3 = ConvBlock(128, 256) # P4 resolution (256 channels)
79         self.pool = nn.MaxPool2d(2, 2)
80         self.bottleneck = ConvBlock(256, 512) # P5 resolution (512 channels)
81
82         # Projection 1x1 convolutions to unify feature channel dimensions to feature_dim
83         self.proj_p3 = nn.Conv2d(128, feature_dim, kernel_size=1)
84         self.proj_p4 = nn.Conv2d(256, feature_dim, kernel_size=1)
85         self.proj_p5 = nn.Conv2d(512, feature_dim, kernel_size=1)
```

```

86         self.proj_p3 = nn.Conv2d(512, feature_dim, kernel_size=1)
87     # BiFPN Decoder Layer
88     self.bifpn = BiFPNDecoderLayer(feature_dim=feature_dim)
89
90     # Segmentation Prediction Head
91     self.seg_head = nn.Sequential(
92         nn.Conv2d(feature_dim, 64, kernel_size=3, padding=1),
93         nn.BatchNorm2d(64),
94         nn.SiLU(),
95         nn.Conv2d(64, 1, kernel_size=1)
96     )
97
98     # Bottleneck Classification Head
99     self.cls_head = nn.Sequential(
100         nn.AdaptiveAvgPool2d((1, 1)),
101         nn.Flatten(),
102         nn.Linear(512, 256),
103         nn.ReLU(),
104         nn.Dropout(0.3),
105         nn.Linear(256, num_classes)
106     )
107
108     def forward(self, x):
109         # Encoder Pass
110         s1 = self.e1(x)
111         p1 = self.pool(s1)
112
113         s2 = self.e2(p1)                # P3 stage
114         p2 = self.pool(s2)
115
116         s3 = self.e3(p2)                # P4 stage
117         p3 = self.pool(s3)
118
119         b = self.bottleneck(p3)         # P5 stage (Bottleneck)
120
121         # Classification Head Output
122         cls_logits = self.cls_head(b)
123
124         # Project features to uniform dimension (feature_dim)
125         p3_feat = self.proj_p3(s2)
126         p4_feat = self.proj_p4(s3)
127         p5_feat = self.proj_p5(b)
128
129         # BiFPN Feature Fusion
130         fused_p3, fused_p4, fused_p5 = self.bifpn(p3_feat, p4_feat, p5_feat)
131
132         # Upsample highest-resolution fused map back to input scale
133         out_seg = F.interpolate(fused_p3, size=(x.shape[2], x.shape[3]), mode='bilinear', align_corners=True)
134         seg_mask = self.seg_head(out_seg)
135
136         return seg_mask, cls_logits
137
138 # --- Verification & Test Execution ---
139 model_bifpn = MultiTaskBiFPNUNet(num_classes=num_classes).to(device)
140 metrics_bifpn = get_full_metrics(model_bifpn, test_loader, device)
141
142 print("=== EFFICIENTDET BiFPN U-NET TEST METRICS ===")
143 display(pd.DataFrame({"BiFPN U-Net Test Set": metrics_bifpn}))

```

=== EFFICIENTDET BiFPN U-NET TEST METRICS ===

BiFPN U-Net Test Set